



Advancement Report

AI-Powered Intrusion Detection System for Cloud Environments

KRID Aboubakr

Master of Artificial Intelligence and Cybersecurity

Supervised by Prof. Fakhri Yousef

May 2026

Contents

1	Context and Continuity	2
2	Track A: Target Services on VM1	3
2.1	Why Real Services Matter	3
2.2	Services Deployed	3
2.3	Verification from VM2	3
3	Track B: Attack Tools and First Attack Tests	4
3.1	Why Kali Linux	4
3.2	Tools Installed and Their Roles	4
3.3	Attack Test 1: SSH Brute Force with Hydra	4
3.4	Attack Test 2: SYN Flood with hping3	5
3.5	What the tcpdump Capture Confirmed	5
4	Track C: Dataset Acquisition	6
4.1	Why CICIDS-2017	6
4.2	What Was Downloaded and What Was Skipped	6
5	ML Environment Setup	7
5.1	Virtual Environment : Why and How	7
5.2	Frameworks Installed	7
5.3	GPU Verification	7
6	Storage Architecture Final Decisions	8
6.1	The Windows Fast Startup Incident	8
6.2	Resolution Dedicated ext4 Partition	8
6.3	Final Storage Layout	9

1. Context and Continuity

The first Explanation Report documented the construction of the lab infrastructure: storage partitioning, Docker migration, KVM virtualisation, and the creation of two virtual machines. At the end of that phase the two VMs could communicate over the isolated `ids-lab` network and both had internet access.

This report picks up from that point and documents three parallel tracks that were completed before model training could begin:

1. **Track A** : Deploying real target services on VM1 that Kali can attack
2. **Track B** : Installing attack tools on VM2 and running the first attack tests
3. **Track C** : Acquiring the CICIDS-2017 dataset and setting up the ML environment

An additional issue arose during this phase, a data loss incident caused by Windows Fast Startup on the NTFS partition, which led to a final storage architecture decision that is also documented here.

2. Track A: Target Services on VM1

2.1 Why Real Services Matter

A core design decision of this project is to use *real* network services as attack targets rather than simulated or synthetic traffic. When Hydra brute-forces a real SSH daemon, the TCP handshakes, authentication exchanges, and timing patterns are genuine. CICFlowMeter extracting features from this traffic produces feature vectors that are directly comparable to the CICIDS-2017 training data, because CICIDS-2017 was itself generated by attacking real services.

This is the principal advantage of the two-VM approach over Mininet simulation.

2.2 Services Deployed

Three services were installed on VM1 (`ids-server`, 192.168.100.46):

Service	Port	Protocol	Attack relevance
Apache2	80	HTTP	HTTP flood, web attacks, DDoS
OpenSSH	22	TCP	Brute force (Hydra), port scan
vsftpd	21	TCP	FTP brute force, port scan

All three services were configured to start automatically on boot using `systemctl enable -now`, ensuring they are always available when VM1 starts without manual intervention.

2.3 Verification from VM2

The services were verified from VM2 (Kali, 192.168.100.28) before any attack testing. The verification confirmed:

- **Apache:** returned HTTP/1.1 200 OK with Apache/2.4.52 server header
- **SSH:** port 22 reported open by netcat
- **FTP:** port 21 reported open by netcat

The `inverse host lookup failed` warning shown in the netcat output is harmless, it simply indicates the absence of a DNS reverse record for the private IP address, which is expected in an isolated lab network.

3. Track B: Attack Tools and First Attack Tests

3.1 Why Kali Linux

Kali Linux is the industry standard for penetration testing and offensive security research. Every tool needed for this project Nmap, Hydra, Metasploit, hping3, Wireshark ..., comes pre-installed and maintained by the Kali team. The alternative would have been manually installing each tool on a generic Ubuntu VM.

The `kali-tools-top10` package installs the ten most widely used offensive security tools in a single command.

3.2 Tools Installed and Their Roles

Tool	Version	Role in project
Nmap	7.99	Port scanning, service fingerprinting
Hydra	9.6	SSH/FTP brute force attacks
hping3	3.0.0	SYN flood, DoS simulation
Metasploit	6.4.126	Exploitation framework
Wireshark	4.6.4	Packet analysis and verification

3.3 Attack Test 1: SSH Brute Force with Hydra

The first attack tested was an SSH brute force against VM1's OpenSSH daemon. Hydra attempted seven passwords against the `akrid` account. The attack produced the following observable behaviour:

- Multiple TCP connections to port 22 in rapid succession
- Each connection completed a full three-way handshake (SYN, SYN-ACK, ACK)
- Hydra identified itself as `libssh_0.11.3` in the SSH banner exchange
- VM1 responded with its own banner: `SSH-2.0-OpenSSH_8.9p1 Ubuntu`

This is the canonical SSH brute force signature: repeated short-duration connections to port 22, each performing a full handshake and authentication attempt, originating from a single IP. This signature appears in the CICIDS-2017 Tuesday dataset and will be a key test case for the trained model.

3.4 Attack Test 2: SYN Flood with hping3

The second attack was a SYN flood against Apache on port 80. hping3 transmitted **5,851,174 packets** in approximately 10 seconds in flood mode. The observable behaviour was:

- Continuous SYN packets with window size 512 (hping3 default) hitting port 80
- VM1 faithfully responding with SYN-ACK to every packet, allocating half-open connection state
- No completed handshakes (no ACK from the attacker)
- RST packets appearing as the connection table overflowed

This is the SYN flood DoS signature. The key distinguishing features are: uniform 40-byte packets, S flag only, no data payload, and an extreme packet rate from a single source IP. This maps directly to the Wednesday DoS dataset in CICIDS-2017.

3.5 What the tcpdump Capture Confirmed

Traffic was captured simultaneously on VM1 during both attacks using `tcpdump` on the `enp1s0` interface. The capture log confirmed that:

- Every attack packet was visible at the network interface level
- The SSH brute force handshakes were fully captured including application-layer banners
- The SYN flood packets were captured at high throughput with correct flag notation
- Source IP 192.168.100.28 (Kali) was correctly identified on every packet

This end-to-end verification attack sent from VM2, captured on VM1 confirms that the traffic capture pipeline works correctly. CICFlowMeter processing these captures will produce valid feature vectors for the detection model.

4. Track C: Dataset Acquisition

4.1 Why CICIDS-2017

The CICIDS-2017 dataset, published by the Canadian Institute for Cybersecurity at the University of New Brunswick, is the standard benchmark for IDS research. It is referenced as [3] in the project’s Advancement Report. Three properties make it appropriate for this project:

1. **Realism:** traffic was generated by 25 simulated users over 5 days using real protocols (HTTP, HTTPS, FTP, SSH, email), not synthetic packet generation
2. **Coverage:** it includes 14 attack categories that correspond directly to the attacks this project simulates in the lab
3. **Feature format:** the CSV files were produced by CICFlowMeter the same tool used to process VM1’s live captures ensuring feature compatibility between training data and live detection

4.2 What Was Downloaded and What Was Skipped

The dataset consists of two forms: raw pcap files (approximately 50 GB) and pre-processed CSV files (approximately 500 MB). Only the CSV files are needed for model training. The raw pcaps have already been processed by CIC; re-processing them would produce identical output.

Of the eight available CSV files, six were downloaded and two were deliberately excluded:

File	Decision	Reason
Monday (Benign)	Downloaded	Essential only purely benign day
Tuesday (Brute Force)	Downloaded	SSH/FTP matches lab attacks
Wednesday (DoS)	Downloaded	Hulk, GoldenEye, Slowloris
Thursday AM (Web Attacks)	Downloaded	SQLi, XSS against Apache
Thursday PM (Infiltration)	Skipped	Very few samples, complex chain
Friday AM (PortScan)	Downloaded	Nmap scan matches lab attacks
Friday PM (DDoS)	Downloaded	Distinct from single-source DoS
Friday PM (Botnet)	Skipped	Extremely few samples, high imbalance

The two skipped files are consistently excluded or reported with near-zero detection rates in published IDS literature. Including them would introduce severe class imbalance without meaningful coverage gain.

5. ML Environment Setup

5.1 Virtual Environment : Why and How

All Python ML frameworks were installed inside a `venv` virtual environment rather than the system Python. This decision has three justifications:

1. **Isolation:** no risk of breaking system Python packages or conflicting with other projects
2. **Reproducibility:** `pip freeze > requirements.txt` captures the exact dependency versions; anyone cloning the repository can reproduce the exact environment with one command
3. **Version control hygiene:** the `venv/` directory is excluded from Git via `.gitignore`, keeping the repository lightweight

5.2 Frameworks Installed

Package	Purpose in project
pandas, numpy	Data loading, cleaning, feature engineering
scikit-learn	Baseline Random Forest, evaluation metrics
torch, torchvision	DNN/LSTM training on RTX 4050 GPU
mlflow	Experiment tracking and model registry
shap	Explainability layer (personal contribution)
matplotlib, seaborn	Evaluation plots, confusion matrices
kafka-python	Streaming pipeline integration
jupyterlab	Exploratory data analysis notebooks

5.3 GPU Verification

The RTX 4050 was confirmed available to PyTorch via CUDA:

Check	Result
<code>torch.cuda.is_available()</code>	True
<code>torch.cuda.get_device_name(0)</code>	NVIDIA GeForce RTX 4050 Laptop GPU

This means neural network training (Phase 3) can be offloaded to the GPU, providing a 5–10× speedup compared to CPU-only training. The Random Forest baseline uses scikit-learn on CPU (RF does not benefit from GPU acceleration).

6. Storage Architecture Final Decisions

6.1 The Windows Fast Startup Incident

During the setup phase, the project folder was initially created on the Windows D: partition (`/dev/nvme0n1p4`, NTFS, mounted at `/media/akrid/D`). After booting into Windows and returning to Linux, the entire project folder had disappeared.

The cause is Windows Fast Startup, enabled by default on virtually all laptops. When Windows “shuts down” with Fast Startup active, it saves the NTFS filesystem state to a hibernation file (`hiberfil.sys`). When Linux subsequently mounts the partition and writes data, those writes are invisible to Windows. When Windows boots again, it restores its saved filesystem state, overwriting all Linux writes. The project folder, created entirely by Linux, was erased.

Why NTFS cannot be used for Linux project data

This is not a bug or a rare edge case it is the designed behaviour of Windows Fast Startup. Any data written to an NTFS partition by Linux while Windows has hibernated is at permanent risk of loss. This applies to all NTFS partitions regardless of whether they were explicitly hibernated by the user.

6.2 Resolution Dedicated ext4 Partition

The resolution was to move all project data to a dedicated ext4 partition. The 37.7 GB of unallocated space discovered between the D: partition and the existing `/mnt/lab` partition was formatted as a new ext4 volume (`/dev/nvme0n1p10`, labelled `lab-project`) and mounted at `/mnt/lab-project`.

The project cannot be directly extended into this free space because it sits to the *left* of the existing `/mnt/lab` partition. Partition extension requires adjacent free space on the right. Rather than performing a risky live partition move (which requires physically relocating all 86 GB of data and takes 2–4 hours with risk of corruption on power loss), two separate mount points were used.

6.3 Final Storage Layout

Partition	Size	FS	Mount	Contents
nvme0n1p3	225 GB	NTFS		Windows C:
nvme0n1p4	31.7 GB	NTFS	/media/akrid/D	Windows D:
nvme0n1p10	35 GB	ext4	/mnt/lab-project	Project code, venv, datasets
nvme0n1p9	79 GB	ext4	/mnt/lab	Docker, VM images
nvme0n1p8	106 GB	ext4	/	Pop!_OS root